

# IaCSecBench: A Finding-Normalization Methodology and Reproducible Harness for Evaluating Infrastructure-as-Code Security Validation

Manideep Chittineni

**Abstract**—Infrastructure as Code (IaC) has become the dominant mechanism for provisioning cloud platforms, yet pre-deployment security validation remains fragmented across static scanners, policy engines, and infrastructure testing frameworks. Comparative evaluation of these tools is impeded by two problems: candidate tools operate at incompatible abstraction levels, from lexical scanning of source templates to policy evaluation over resolved execution plans, and each reports findings in a bespoke schema. Benchmarks authored by tool developers compound the difficulty, since the same interpretation of a security control tends to define both the policy and the ground truth. This paper presents IaCSecBench, an open harness for evaluating layered IaC security validation pipelines, together with a finding-normalization methodology that maps heterogeneous scanner output onto a canonical control taxonomy and thereby permits confusion-matrix comparison across tools with unrelated report formats. The harness composes repository-edge data validation, native Terraform module testing, and policy evaluation over compiled plan documents; the first and third are scored per case, and the second is reported by suite size only. Over a labelled corpus of 48 admissible cases we evaluate three third-party scanners, spanning two independent rule sets (one is the maintained successor of another and inherits its rules), against the pipeline’s own layers and their union, and assess generalization separately on independently sourced configurations. Two results are the substance of the paper. The matching criterion moves apparent recall further than the differences between tools do, by up to a factor of 24 on one layer, so a comparison that does not state its criterion is not reproducible at any corpus size. And the reference pipeline does not lead the comparison: a third-party scanner attains the highest control-level recall, which we report as measured. Detection effectiveness is reported with exact Clopper-Pearson intervals, and pairwise differences with exact binomial tests under family-wise correction; no pairwise difference survives correction, which the interval widths explain. We further report a mechanically enforced corpus admissibility procedure that rejects cases which cannot support a defensible ground-truth label. The corpus, normalization engine, execution harness, and analysis code are released as a replication package.

**Index Terms**—Cloud computing security, continuous integration, DevSecOps, Infrastructure as Code, policy as code, software benchmarking, static analysis.

## I. Introduction

CLOUD platforms are increasingly provisioned through Infrastructure as Code (IaC), in which infrastructure topology is expressed declaratively in version-controlled artefacts [1], [2], atop service models whose

elasticity makes such automation a practical necessity [3]. The property that makes IaC attractive, namely deterministic and repeatable application of a declared state, also amplifies its failure modes: a single insecure declaration, such as an unrestricted ingress rule or a disabled encryption setting, propagates automatically to every environment that consumes the module. Empirical studies of configuration repositories confirm that such defects are common, not exceptional, recurring as hard-coded secrets, excessive privileges, and insecure defaults [4], [5], [6], [7].

Because remediation after provisioning necessarily implies a window of exposure, contemporary practice shifts validation earlier, into continuous integration [8], [9]. Doing so raises an evaluation problem the literature has not resolved. Candidate gates operate at incomparable abstraction levels: lexical scanning of HCL source, abstract-syntax tree analysis, and policy evaluation over resolved plan state. Each reports findings in a bespoke schema, with rule identifiers that carry no shared semantics. Comparative claims in this space are consequently difficult to reproduce, and when a benchmark is authored by the developer of one of the tools under comparison, agreement between that tool and the ground truth is close to guaranteed by construction.

This paper addresses the evaluation problem; it does not propose a further analyser. Three commitments shape the design.

First, normalization before comparison. A canonical control taxonomy mediates between native rule identifiers and benchmark ground truth, so that a finding is credited when it identifies the control a case was authored to violate, and not merely when the tool emitted something. Findings whose rule identifiers the taxonomy does not cover are counted and reported, never discarded, because silently treating an unmapped detection as a miss biases the comparison toward whichever tool the taxonomy was authored around.

Second, admissibility is mechanically enforced. A case is admitted only if it is valid HCL2 that references resource types the declared provider defines, carries a non-contradictory ground-truth label, and resolves to an unambiguous canonical control. The reported corpus size is the admissible count, not the number of entries in a catalogue.

Third, measurement is separated from assumption. A tool that is not installed in the measurement environment

M. Chittineni is a Senior Platform, Cloud and DevOps Engineer, United Kingdom (e-mail: manideep.chittineni@hotmail.com).

is reported as such and excluded; it is never represented by an assumed detection rate. Latency is sampled repeatedly and reported with dispersion.

#### A. Contributions

- A finding-normalization methodology and canonical control taxonomy that render heterogeneous IaC scanner output commensurable, with three explicit matching strictness levels that separate correct attribution from incidental detection.
- A corpus admissibility procedure that enforces the stated inclusion criteria mechanically, together with the measured admissibility of the released corpus.
- An execution harness that records raw tool output, resolved tool versions, and repeated latency samples, and that refuses to emit results when no case is admissible. Applied to its own reference pipeline, the harness reports that a third-party scanner detects more of the corpus than the pipeline's scored layers do, which is the outcome we publish.
- A statistical protocol appropriate to paired, small-sample scanner comparison: exact Clopper-Pearson intervals [10], the exact binomial form of McNemar's test [11], [12], and Holm-Bonferroni correction across the tool family [13].
- An open replication package containing the corpus, taxonomy, harness, and analysis code, in which every reported table is a generated artefact.

## II. Background and Related Work

### A. Security Weaknesses in Infrastructure as Code

Rahman et al. established that infrastructure scripts contain recurring security smells, first across Puppet [4] and subsequently in Ansible and Chef [5], and derived a defect taxonomy for IaC scripts [14]. Complementary work characterises misconfiguration in Kubernetes manifests [15], [16] and security practice in Terraform repositories specifically [6], [7]. Quality and defect-prediction metrics for infrastructure code have also been catalogued [17], [18]. This body of work motivates automated pre-deployment validation but does not address how competing validation tools should be compared.

### B. Static Analysis of IaC and Existing Comparisons

Chiari et al. survey static analysis for IaC and note the absence of shared evaluation infrastructure [19]. The closest detector-side prior art is GLITCH, which detects security smells across several IaC technologies via an intermediate representation and is evaluated against a labelled oracle dataset with a tool comparison [20], [21]. IaCsecBench differs in target and in claim: GLITCH contributes a detector with polyglot coverage, whereas this work contributes the normalization and admissibility methodology needed to compare detectors that report incommensurable findings, and applies it to plan-level as well as source-level evaluation. The taxonomy those

detectors are scored against is itself under revision, most recently beyond the original seven smell categories [22], which is one reason this paper versions its control set as data rather than fixing it in prose.

Comparative evaluation of the practitioner scanners has begun to appear. Roe et al. benchmark IaC scanners across multiple cloud providers and report per-tool detection counts [23]; Gokhale et al. contrast policy-driven evaluation with taint-aware static reasoning for IaC misconfiguration [24]; and Kapetanidou et al. evaluate the equivalent tool class for Kubernetes [25]. These establish that the comparison is wanted; none addresses the two obstacles this paper targets. Findings are compared in whichever schema each tool emits, so a rule that fires for the wrong reason is indistinguishable from one that fires for the right one, and the corpora are not released with an admissibility procedure that a reader can re-run. Our contribution is orthogonal to theirs and could be applied to their corpora.

Opdebeeck et al. show that control- and data-flow analysis materially changes security-smell detection in IaC [26], [27]. That result motivates the plan-level layer evaluated here: a compiled Terraform plan is a resolved artefact in which interpolation, conditionals and expansion have already been applied, so evaluating it sidesteps a class of analysis difficulty instead of solving it. Qiu et al. pursue the complementary route of recovering semantic checks over IaC programs directly [28], and policy-violation analysis has been extended to packaged Kubernetes artefacts [29]. Empirical work on policy-as-code adoption indicates which controls practitioners actually enforce [30].

Methodologically, this paper follows the static-analyser evaluation tradition established by Habib and Pradel, who show that headline detection rates are highly sensitive to what counts as a match [31], and the benchmark-construction cautions of the OWASP Benchmark project [32]. The three-level matching criterion in §III-D is a direct response to the first. The reporting conventions we adopt are those of the SAST tool-comparison literature, which reports per-category rather than pooled effectiveness and separates the absence of a rule from the failure of one [33], [34]. Bohme et al. establish the wider point that benchmark results fail to replicate when configuration and variance go unreported [35], which is the argument for the version manifest of Section V-A and for preferring interval width to corpus size as an indicator of evidential strength.

### C. Policy as Code and Native Infrastructure Testing

Open Policy Agent provides general-purpose policy evaluation over structured documents [36], and Terraform provides a native test harness for module behaviour [37]. Compliance requirements are drawn from the CIS AWS Foundations Benchmark [38] and NIST SP 800-53 [39]. IaCsecBench composes these mechanisms; it does not extend them.

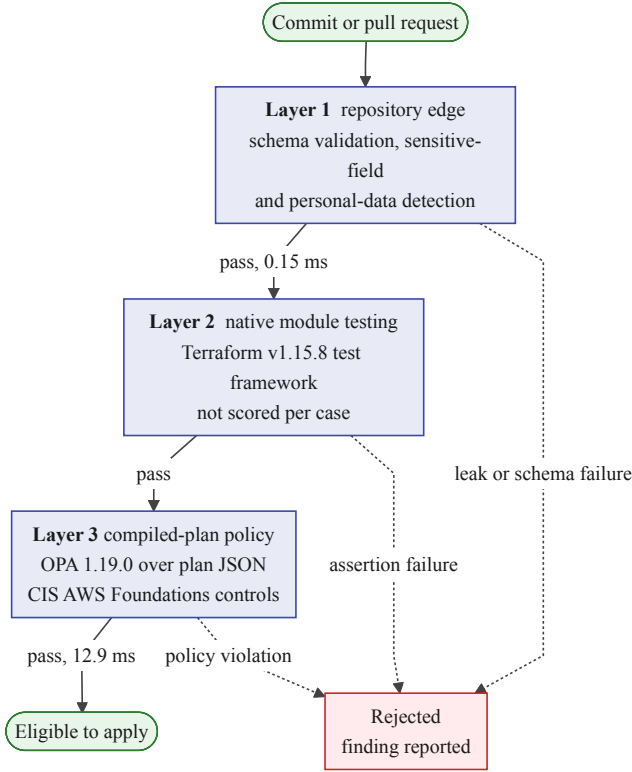


Fig. 1. Layered validation pipeline: repository-edge data validation, native module testing, and policy evaluation over compiled plan documents. A change reaches a later layer only by passing every earlier one. Edge labels give the mean per-case latency of the scored layers; the full figures, including dispersion, are in Table VII. Layer 2 validates module behaviour rather than resource configuration, so it carries no detection count on this corpus and is reported by suite size in Section VI-C.

### III. Framework Architecture

IaCSecBench composes three validation layers, each addressing a distinct failure class. Fig. 1 shows their arrangement and the gating relation between them: a commit reaches a later layer only by passing every earlier one, so the layers compose as a conjunction rather than as independent checks.

Two of the three are scored per benchmark case. Layer 2 validates module behaviour rather than detecting resource misconfiguration, so it has no detection count on a corpus of misconfigured resources and is reported by suite size alone in Section VI-C. Where this paper reports the pipeline’s detection effectiveness, the figure is the union of layers 1 and 3, and we mark it as such rather than let a three-layer architecture imply three measured layers.

#### A. Layer 1: Repository-Edge Data Validation

The first layer prevents sensitive material from entering version control through schema verification, detection of sensitive fields, and pattern-based identification of personal data. This layer operates on repository content rather than infrastructure semantics, and its characteristic failure mode is over-detection: high-entropy strings in

output declarations are frequently flagged as credentials. Over-detection is not a cosmetic cost. Developers discount tools whose warnings they learn to distrust, and the effect is documented both for static analysis generally [40] and for security warnings specifically [41], to the point that identifying which warnings merit action is now a research area in its own right [42]. This layer’s false-positive behaviour is therefore reported separately in §VI and is not aggregated with the policy layers.

#### B. Layer 2: Native Module Testing

The second layer validates module behaviour using Terraform’s native test framework [37], covering variable constraints, output correctness, conditional resource creation, and tagging requirements without provisioning cloud resources.

#### C. Layer 3: Policy Evaluation over Compiled Plans

The third layer converts a Terraform plan to its JSON representation and evaluates Rego policies against it [36]. Because the plan is produced after expression resolution, policies observe concrete attribute values, not unevaluated references. Plan generation is configured so that provider credential validation is disabled, which permits evaluation in continuous integration without cloud account access; this is a precondition for the layer being usable at all, and is implemented by an injected provider override, never by relaxing a policy.

#### D. Finding Normalization

Comparing heterogeneous scanners requires a canonical representation. A finding is normalised to

$$\mathcal{F} = \langle c, t, \kappa, r, s \rangle, \quad (1)$$

where  $c$  identifies the benchmark case,  $t$  the tool,  $\kappa$  the set of canonical controls the tool’s native rule maps to,  $r$  the resource address, and  $s$  the reported severity. The mapping from native rule to  $\kappa$  is many-to-many: a coarse rule may legitimately cover several controls, and such rules are reported explicitly because they weaken attribution precision.

Detection is then evaluated at three strictness levels:

- control: some finding’s  $\kappa$  intersects the controls the case was authored to violate. This is the primary criterion.
- resource: some finding names the resource address the ground truth identifies, irrespective of which rule fired. This isolates the correct-resource-wrong-reason case.
- any: the tool emitted any finding within the case. This criterion overstates detection and is reported only to quantify how much of a tool’s apparent performance is incidental.

Reporting all three is deliberate: adopting a single permissive criterion is the most common mechanism by which scanner benchmarks overstate detection [31]. Where the

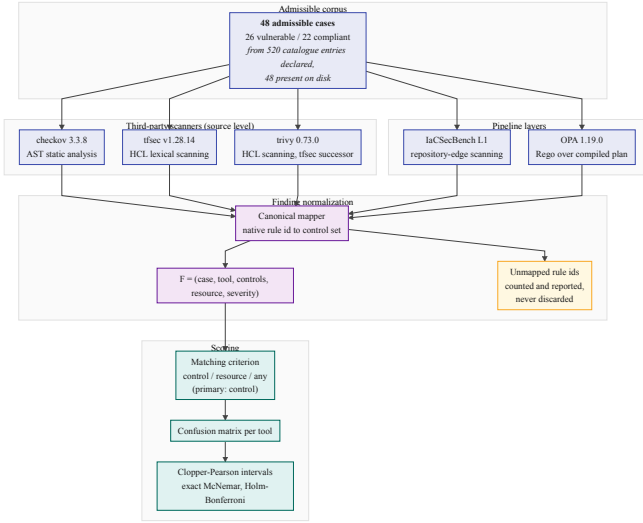


Fig. 2. Finding-normalization engine and evaluation pipeline: heterogeneous scanner output is mapped onto the canonical control taxonomy before any confusion matrix is formed.

ground truth does not name a resource address, the resource criterion is inapplicable and is reported as such, not as a failure.

Fig. 2 traces a finding through the engine. The ordering matters for the comparison’s validity: normalization onto the canonical taxonomy precedes confusion-matrix construction, so no tool is scored in its own reporting schema and the matching criterion is applied identically to every tool.

#### IV. Scope of the Validated Threats

The controls the pipeline enforces were selected against a STRIDE decomposition [43] of a continuous-integration deployment path, summarised in Table I. Protected assets are infrastructure configurations and compiled plan states, ingested datasets, policy and test definitions, and pipeline credentials; the trust boundaries separate the developer workspace from version control, the evaluation runtime from the target repository, and the compiled plan from the policy engine.

We record the decomposition to delimit which misconfiguration classes the corpus exercises, and claim nothing further from it. The pipeline was not subjected to adversarial testing, no adversary model was validated, and the table is a statement of intended coverage rather than a result. Readers seeking a threat model of IaC pipelines as such should look to dedicated work [16]; readers wanting to know what this evaluation measured should read the corpus coverage of Section V-B instead, which is the operative scope statement.

Four domains are out of scope: post-deployment runtime compromise, including container and hypervisor vulnerabilities; supply-chain compromise of provider binaries or registry mirrors; IaC technologies other than Terraform, since the plan-level layer presupposes Terraform plan JSON; and physical and social-engineering attacks.

TABLE I  
STRIDE categories, mitigations, and the layer that enforces them.

| Category               | Mitigation                                                                                           | Layer |
|------------------------|------------------------------------------------------------------------------------------------------|-------|
| Spoofing               | Authenticated API authorisers; transport encryption enforcement; prohibition of embedded credentials | 1, 3  |
| Tampering              | Schema validation; pre-commit enforcement; evaluation of compiled plan documents                     | 1, 2  |
| Repudiation            | Structured pipeline telemetry; recorded test execution; compliance tagging                           | 2, 3  |
| Information disclosure | Pattern-based personal-data detection; mandatory encryption at rest; storage public-access blocking  | 1, 3  |
| Denial of service      | Validation of web application firewall association, header handling, and request throttling          | 2, 3  |
| Elevation of privilege | Policy denial of wildcard identity actions and unrestricted ingress ranges                           | 3     |

#### V. Evaluation Methodology

##### A. Corpus Construction and Admissibility

The corpus separates a controlled collection, authored for this evaluation, from an external collection sourced independently. The distinction matters because the controlled collection shares an author with the policy set and therefore cannot provide independent confirmation of detection capability.

Admissibility is enforced mechanically; it is not asserted. A case is admitted only if it satisfies all of the following:

- 1) it declares at least one resource or module block;
- 2) its ground-truth label is present and internally consistent, where a case declaring contradictory labels is rejected, not resolved by precedence;
- 3) it resolves to exactly one interpretation in the canonical taxonomy, since a CIS section number may span several distinct controls and inference cannot choose between them;
- 4) terraform init and terraform validate succeed, which is the only check that detects a case referencing a resource type the provider does not define.

Table II reports the outcome of applying this procedure. The admissible count is the corpus size used throughout; catalogue entries without corresponding configuration are not cases and are excluded. The rejection reasons characterise the corpus more faithfully than a headline size does, and are given in full.

TABLE II

Corpus admissibility. The admissible count is the corpus size reported throughout; catalogue entries without configuration are not cases. Rejection reasons are determined mechanically by `evaluation/corpus.py`.

| Corpus status                  | Cases |
|--------------------------------|-------|
| Catalogue entries declared     | 520   |
| Configurations present on disk | 48    |
| Admissible                     | 48    |
| vulnerable                     | 26    |
| compliant baseline             | 22    |

## B. Control Coverage

The canonical taxonomy defines 26 controls, of which 22 are exercised by at least one admissible case. The two numbers answer different questions: the size of the taxonomy describes what the normalization engine can express, while the exercised count describes what this evaluation measured. Reporting only the former would credit the corpus with four controls (one for API-gateway authorization and three for Kubernetes workloads) against which nothing was ever run.

The unexercised controls are concentrated in the domains the corpus does not reach at all: Kubernetes workloads and serverless authorization. Their absence is a coverage limitation of the corpus, not a negative result about any tool, and no tool is penalised for them, since a control with no case contributes to no confusion matrix.

Auditing the taxonomy against the installed tool catalogues found three defect classes in the mapping itself, all corrected. Seven controls named plan-level policy rules the policy set does not implement. Two named Checkov identifiers absent from the installed catalogue entirely, and one mapped a resource-requests check to a resource-limits control, which are distinct Kubernetes concepts. None of these can produce a spurious detection, because nothing emits the identifier; each instead inflates the apparent coverage of the layer it is attributed to, and in the first case that layer is the reference implementation. That is the direction of error that flatters the contribution under evaluation, which is why the audit was mechanical rather than by inspection, and why the resulting invariant, that every plan-level identifier must exist in the policy source, is now enforced by a test.

The correction left two controls with no implementing rule in any tool, and so detectable by none of them. A control in that state scores every case citing it as a miss for every tool simultaneously, which reads as unanimous tool failure rather than as absent coverage; both were removed, and neither was exercised, so no reported figure changes. Eight controls retain an unverified flag, each naming the single identifier behind it. All eight are tfsec long identifiers that this environment cannot corroborate: tfsec 1.28 provides no check-listing command, and its `-filter-results` option returns nothing for an unknown identifier and for a real but untriggered one alike. They are retained

rather than deleted, because a wrong identifier is inert (nothing emits it, so it never matches), and the only possible effect of keeping it is to credit a comparator if it turns out to be real. Where the two options differ, we take the one that cannot flatter the reference.

## C. Ground-Truth Labelling

Labels are derived mechanically, not by human rating. Each controlled case is emitted by a generator from a per-control specification that declares the intended violation, and the label follows from which member of the vulnerable/compliant pair is being written. The generator, the specification, and the emitted cases are all in the replication package, so any reader can check that a case carries the label its specification declares.

Because labels are a deterministic function of the specifications, rater disagreement is impossible by construction, and the threat that replaces it is specification error: a specification that misreads its originating control produces a wrong label reproduced identically across the whole pair. An agreement coefficient computed over the generator's own output would measure its determinism, not concurrence, so we do not report one.

We instead ran a blind second labelling pass against the threat that does apply. For each admissible case a review view was constructed containing only the Terraform configuration and the canonical control under test, with every comment line removed, since the generator writes the expected label and its rationale into each file header, and with the annotation files and the case identifier withheld, the latter because its suffix names the class. Cases were presented in an order keyed on a hash of the case identifier, and the views were screened for residual label vocabulary. Each was then labelled from the configuration and the control text alone, with a reason recorded before any comparison.

Agreement is reported before reconciliation, since resolving disagreements guarantees consensus: 47 of 48 cases agreed, giving raw agreement of 97.92% and Cohen's  $\kappa = 0.958$  against a chance-agreement baseline of 50.17%.

The single disagreement is instructive, and it is the kind of defect this pass exists to surface. A bucket configured with SSE-S3 (AES256) was labelled violating under a control titled lacks server-side encryption and citing CIS Amazon Web Services 2.1.1. That configuration does not lack server-side encryption, and CIS 2.1.1 is satisfied by SSE-S3; read against its stated text the case is compliant. The generator's rationale reveals the intent, namely encryption other than a customer-managed key, so the label encoded a requirement the control text never stated. That stricter requirement is also precisely what the reference policy set enforces, which makes this a concrete instance of the construct-validity threat of Section VIII-A and not a hypothetical one. The ground truth agreed with the reference implementation rather than with the control it cited.

We corrected the control text rather than the label, because requiring a customer-managed key is a defensible

control and is what the corpus, the policy set, and the customer-key rules of both source-level scanners all test; relabelling instead would have left the pair testing nothing. The control is retitled to state the key requirement and records that it is stricter than the CIS control it cites. No confusion-matrix entry changed. A regression test now pins the citation in the taxonomy to the one in the generator specification so the two cannot drift apart silently.

Two properties of this pass bound what it establishes, and we state them rather than let  $\kappa$  speak unqualified. The second rater is an automated one, so this is not human inter-rater reliability; and its reading of the control texts derives from the same public documentation the specifications were written from, so the two raters are not fully independent and agreement is biased upward accordingly. Five cases had been inspected during earlier debugging and their agreement is not blind; excluding all five leaves 42 of 43 agreeing, with the disagreement outside that set. The full per-case labels, reasons, and these caveats are in the replication package.

Two further checks bear on specification error. The admissibility procedure of Section V-A rejects any case whose label is absent, internally contradictory, or resolvable to more than one control in the taxonomy. And the audit of the control map reported in Section V-B found three classes of mapping defect by mechanical comparison against the installed tool catalogues, which is the same style of check applied to the other half of the scoring path. Neither check can establish that a specification reads its control correctly; that remains an unmitigated threat, and independent relabelling by a second investigator is the obvious next step.

#### D. Research Questions

- RQ1: What detection effectiveness do the evaluated tools achieve on the admissible controlled corpus, and how precisely is that effectiveness estimated?
- RQ2: How much does the matching criterion change apparent detection?
- RQ3: What execution latency does each layer contribute in continuous integration, and what is the size of the native validation suite required to express the corresponding assertions?
- RQ4: Which layers account for which detections?
- RQ5: Does effectiveness persist on independently sourced configurations?

#### E. Measurement Environment and Tool Versions

Tool versions are resolved at execution time and recorded in `results/run_manifest.json`, together with the platform, processor, and repository commit. Versions are not restated in prose, because a hand-copied version string is the first thing to drift; the manifest is authoritative.

Tools that are not installed in the measurement environment are recorded as such and omitted from every table. No tool is assigned an assumed detection rate or an assumed latency.

#### F. Tool Selection

The comparison covers Checkov, tfsec, Trivy, and standalone policy evaluation over plan documents. Selection was governed by two criteria: the tool must parse HCL2 or Terraform plan JSON without requiring cloud provider API access, and the set must span distinct analysis paradigms: syntax-tree analysis, lexical scanning, and policy evaluation. KICS, Regula, and provider-native configuration services are excluded under the first criterion or as duplicating a represented paradigm.

Trivy requires separate comment, because it is in the set for a reason unrelated to paradigm coverage and its inclusion would otherwise be misread as adding an independent opinion. Aqua Security folded tfsec's engine and rule set into Trivy, and Trivy is the maintained product; tfsec is no longer developed. A comparison that reported tfsec alone would be characterising a scanner practitioners are advised off, so both are measured. But the two are not independent implementations. Their rule identifiers are the same Aqua Vulnerability Database numbers under different spellings (tfsec reports AVD-AWS-0086 where Trivy reports AWS-0086), and we exploit exactly that correspondence to build Trivy's entries in the control taxonomy: each is derived mechanically from the identifier pair tfsec itself emits, rather than chosen by observing which rules Trivy happens to fire on a case, which would fit the taxonomy to the tool. A regression test re-derives the crosswalk and fails if the two disagree.

The consequence bounds what the enlarged tool set establishes. Three third-party scanners are measured, but only two independent third-party rule sets: Checkov's, and the tfsec/Trivy lineage. Agreement between tfsec and Trivy is therefore close to a consistency check on that shared lineage across two engine generations, not corroboration by a second party, and we do not report it as the latter. Where the two diverge, the difference is attributable to engine and rule evolution between the products and is informative for that reason alone.

Terrascan is absent for a reason that is not a selection criterion, and we state it plainly rather than present the paradigm argument as the cause: no successful Terrascan execution was ever recorded against this corpus, so there is nothing to report. Its paradigm, Rego policy evaluation, is represented in the tool set by the plan-level layer, which bounds what the omission costs but does not eliminate it. Policy evaluation is therefore represented only over compiled plan documents, and no source-level policy engine appears in the tool set. A source-level Rego engine could differ from a plan-level one on configurations whose violations are resolved during planning, and this evaluation does not measure that difference. We make no claim of any kind about Terrascan's detection behaviour.

#### VI. Results

All tables in this section are generated by `evaluation/analyze.py` from recorded raw tool output. They are regenerated by `experiments/run_baselines.sh` and are not edited by hand.

### A. RQ1: Detection Effectiveness

Table III reports confusion matrices with exact Clopper-Pearson intervals. Intervals are reported for every proportion because a point estimate on a corpus of this size is compatible with a wide range of true values; a perfect point estimate in particular carries a lower confidence bound substantially below unity, and reporting the point estimate alone would misstate what has been established.

The Clopper-Pearson form is chosen for its coverage guarantee rather than for interval width. Its actual coverage is never below the nominal level, whereas the score and adjusted-Wald intervals attain nominal coverage only on average and undercover for proportions near the boundary [44], which is where most of the estimates here sit. The cost is conservatism, so every interval in Table III is wider than a score interval on the same counts would be. For a claim about detection capability that is the safe direction of error, and the harness computes the score interval as well, so a reader preferring it can obtain it from the released results.

One property of the precision intervals should be read with care. Recall's denominator is the 26 vulnerable cases, fixed by the corpus design, so a binomial interval applies directly. Precision's denominator is the number of findings the tool emitted, which is a realised outcome rather than a design parameter, so its interval is conditional on the observed prediction count and is not a marginal statement about the tool.

Metrics that the corpus cannot support are marked as not estimable instead of printed as zero. Specificity and the Matthews correlation coefficient [45] require ground-truth-compliant cases, and in their absence no false-positive behaviour is characterised at all.

Table IV reports pairwise comparisons. The discordant count  $b$  includes every case in which the reference is correct and the comparator is not, counting spurious findings as well as missed violations; restricting  $b$  to missed violations understates disagreement. The exact binomial  $p$ -value is reported in preference to the chi-square approximation, which is unreliable when a discordant cell is small and is therefore not reported as a primary result.  $p$ -values are corrected across the tool family [13]; test selection follows established guidance for software-engineering experiments [46]. Odds ratios carry a Haldane-Anscombe correction, so a zero discordant cell yields a finite estimate; an unbounded ratio is a degenerate cell, not a large effect, and Cohen's  $g$  is reported as the effect size.

No pairwise difference among the three third-party scanners and the plan-level layer is significant after correction. This is the expected consequence of the corpus size, not evidence of equivalence. With 26 violating cases the discordant cells are small enough that only a large difference could be resolved, and the exact test correctly declines to resolve differences of one or two cases. The interval widths in Table III are the honest summary of what this corpus establishes about relative effectiveness,

which is little.

Comparing every tool against one reference leaves two things unaddressed: the comparison a practitioner most wants, between two third-party scanners, is never made, and the tool this paper contributes sits on one side of every test. Table V therefore reports all ten pairs, with Holm-Bonferroni applied across the whole set rather than across one tool's family. The correction is correspondingly stronger, which is the cost of asking more questions of the same corpus rather than a reason to ask fewer. The conclusion is unchanged: every comparison not involving the repository-edge layer is non-significant after correction, and the repository-edge layer differs from all four others because it addresses a different class of defect entirely, which is a statement about scope and not about quality.

1) Divergence within the shared rule lineage: The tfsec/Trivy pair (Section V-F) permits one measurement the rest of the comparison cannot make: how much a rule set changes as it is carried from a retired product into its successor. The two agree on 46 of 48 cases. Both disagreements are the same control, IAM policy wildcards: tfsec reports `aws-iam-no-policy-wildcards` on both members of the pair, and Trivy emits its counterpart `AWS-0057` on neither: not on these two cases, and nowhere in the corpus.

The divergence is not a straightforward regression. tfsec's rule fires on the violating case, earning a true positive, and on the compliant case, earning a false positive; Trivy's silence forfeits both. tfsec therefore records the higher recall and Trivy the higher precision, from a single rule behaving differently. We verified that Trivy's miss is genuine and not an artefact of our taxonomy. The one unmapped finding Trivy emitted on that case is an `S3` versioning rule, unrelated to IAM policy wildcards, so no mapping gap is concealing a detection.

The remainder of the two tools' output corroborates the shared provenance quantitatively. Of the rule identifiers each emitted that the taxonomy does not cover, 12 of tfsec's 13 have an exact Trivy counterpart reported on an identical number of cases; the thirteenth, a `CloudFront` `TLS-policy` rule, is absent from Trivy as `AWS-0057` is. Two products separated by a major version and a change of engine thus differ, on this corpus, by two rules. Practitioners migrating from the retired scanner to its replacement should not assume that coverage transfers rule for rule, and a benchmark that measured only one of the two would have attributed the difference to nothing.

### B. RQ2: Sensitivity to the Matching Criterion

Table VI reports recall under each criterion. The spread between the control and any columns measures how much apparent detection is attributable to findings unrelated to the seeded violation. A tool whose recall is high under the permissive criterion and substantially lower under the control criterion is emitting findings on these cases without identifying the vulnerability in question.

TABLE III

Detection performance on the admissible corpus ( $N = 48$ : 26 vulnerable, 22 compliant). Intervals are exact Clopper–Pearson at the 95% level. Entries marked n/e are not estimable from this corpus.

| Tool             | TP | FP | TN | FN | Rec. (%) | Rec. 95% CI    | Prec. (%) | Prec. 95% CI    | MCC    |
|------------------|----|----|----|----|----------|----------------|-----------|-----------------|--------|
| Checkov          | 23 | 0  | 22 | 3  | 88.46    | [69.85, 97.55] | 100.00    | [85.18, 100.00] | 0.882  |
| IaCSEC Bench L1  | 1  | 1  | 21 | 25 | 3.85     | [0.10, 19.64]  | 50.00     | [1.26, 98.74]   | -0.017 |
| OPA (plan-level) | 20 | 0  | 22 | 6  | 76.92    | [56.35, 91.03] | 100.00    | [83.16, 100.00] | 0.777  |
| tfsec            | 22 | 2  | 20 | 4  | 84.62    | [65.13, 95.64] | 91.67     | [73.00, 98.97]  | 0.753  |
| Trivy            | 21 | 1  | 21 | 5  | 80.77    | [60.65, 93.45] | 95.45     | [77.16, 99.88]  | 0.762  |

TABLE IV

Exact McNemar comparisons against OPA (plan-level).  $b$  counts cases the reference classifies correctly and the comparator does not, counting both missed violations and spurious findings;  $c$  is the converse.  $p$  is the two-sided exact binomial value,  $p_{adj}$  its Holm–Bonferroni correction across the tool family. The odds ratio carries a Haldane–Anscombe correction so that a zero discordant cell yields a finite estimate.

| Comparator      | $b$ | $c$ | $p$     | $p_{adj}$ | OR    | Cohen's $g$ |
|-----------------|-----|-----|---------|-----------|-------|-------------|
| Checkov         | 2   | 5   | 0.453   | 1.000     | 0.45  | 0.214       |
| IaCSEC Bench L1 | 21  | 1   | < 0.001 | < 0.001   | 14.33 | 0.455       |
| tfsec           | 3   | 3   | 1.000   | 1.000     | 1.00  | 0.000       |
| Trivy           | 3   | 3   | 1.000   | 1.000     | 1.00  | 0.000       |

TABLE V

Exact McNemar comparisons over every tool pair, with Holm–Bonferroni applied across the full set of pairs rather than across one tool's family.  $b$  counts cases the left tool classifies correctly and the right does not;  $c$  is the converse. Both error types count. The correction is stronger here than in Table IV because the family is larger.

| Tool A           | Tool B           | $b$ | $c$ | $p$     | $p_{adj}$ |
|------------------|------------------|-----|-----|---------|-----------|
| Checkov          | IaCSEC Bench L1  | 24  | 1   | < 0.001 | < 0.001   |
| Checkov          | OPA (plan-level) | 5   | 2   | 0.453   | 1.000     |
| Checkov          | tfsec            | 4   | 1   | 0.375   | 1.000     |
| Checkov          | Trivy            | 4   | 1   | 0.375   | 1.000     |
| IaCSEC Bench L1  | OPA (plan-level) | 1   | 21  | < 0.001 | < 0.001   |
| IaCSEC Bench L1  | tfsec            | 3   | 23  | < 0.001 | < 0.001   |
| IaCSEC Bench L1  | Trivy            | 2   | 22  | < 0.001 | < 0.001   |
| OPA (plan-level) | tfsec            | 3   | 3   | 1.000   | 1.000     |
| OPA (plan-level) | Trivy            | 3   | 3   | 1.000   | 1.000     |
| tfsec            | Trivy            | 1   | 1   | 1.000   | 1.000     |

TABLE VI

Recall under three finding-matching criteria. Control credits a detection only when the tool's rule maps to the control the case violates; resource credits any finding on the expected resource; any credits any finding whatsoever. The spread measures attribution precision.

| Tool             | Control (%) | Resource (%) | Any (%) |
|------------------|-------------|--------------|---------|
| Checkov          | 88.46       | 76.92        | 96.15   |
| IaCSEC Bench L1  | 3.85        | 0.00         | 92.31   |
| OPA (plan-level) | 76.92       | 65.38        | 84.62   |
| tfsec            | 84.62       | 73.08        | 92.31   |
| Trivy            | 80.77       | 69.23        | 92.31   |

TABLE VII

Measured per-case execution latency over 3 repetitions. Values are mean  $\pm$  standard deviation on the environment recorded in results/run\_manifest.json. Each tool is measured at its own interface: the plan-level figure covers policy evaluation over an already-compiled plan document and excludes the terraform init and terraform plan invocations that produce it, which dominate that layer's wall-clock cost in continuous integration. It is therefore a lower bound, and is not comparable with the source-level figures, which have no equivalent preparation step.

| Tool             | Evaluation layer              | Latency (ms) |      |
|------------------|-------------------------------|--------------|------|
| Checkov          | AST static analysis           | 1853.4 $\pm$ | 58.9 |
| IaCSEC Bench L1  | Repository-edge scanning      | 0.1 $\pm$    | 0.1  |
| OPA (plan-level) | Rego over compiled plan       | 12.9 $\pm$   | 5.8  |
| tfsec            | HCL lexical scanning          | 275.4 $\pm$  | 8.4  |
| Trivy            | HCL scanning, tfsec successor | 405.5 $\pm$  | 32.0 |

### C. RQ3: Execution Cost and Suite Size

Table VII reports per-case latency as a mean with standard deviation over repeated executions. Single-run figures are not reported. Variance on shared-tenancy runners is large enough that a one-decimal single-sample figure is not a measurement.

On engineering effort we report a measurement of the native suite alone and withdraw the comparative claim. The validation suite comprises 11 .tftest.hcl files totalling 190 physical lines, of which 153 are non-blank and 137 are both non-blank and non-comment, expressing 11 run blocks and 26 assert blocks. We give both line counts because they differ by the 16 comment lines and one of them is the figure a reader would compute; quoting the larger under the stricter description would inflate the suite by 12%. Counts were obtained by line-oriented pattern matching over the suite sources, and the sources are in the replication package so the count can be reproduced or disputed.

We do not report a code-volume ratio against an external test framework. Doing so would require an independently authored equivalent suite, and no such implementation exists for this infrastructure. Constructing one ourselves would place both sides of the comparison in the hands of the party with an interest in the outcome, which is not a measurement we would ask a reader to accept. The claim that native testing reduces test-suite volume relative to an external framework is therefore outside the scope of this paper. We note only that the native suite

TABLE VIII

Layer attribution over the vulnerable cases. Overlap is reported explicitly: layers are not assumed to detect disjoint sets. Layer 2 (native module testing) is not scored: it validates module behaviour rather than detecting resource misconfiguration, so it has no detection count on this corpus.

| Configuration          | Detected | Recall (%) |
|------------------------|----------|------------|
| L1 (repository edge)   | 1        | 3.85       |
| L3 (compiled plan)     | 20       | 76.92      |
| $L1 \cap L3$ (overlap) | 0        | 0.00       |
| Union of scored layers | 21       | 80.77      |

requires no language runtime, dependency manifest, or build step beyond the Terraform binary already needed to plan the configuration. That is a qualitative difference in toolchain surface, not a quantitative claim about effort.

#### D. RQ4: Layer Attribution

Table VIII attributes detections to the layer that produced them. Each layer is scored by the same normalization and matching procedure applied to the external scanners, so a layer is credited only when its finding maps to the control the case seeds; no layer receives credit by construction.

The table reports pairwise intersections and the union alongside the per-layer counts, because the per-layer counts alone cannot distinguish a complementary pipeline from a redundant one. If the layers were reported additively, a case detected by every layer would be counted repeatedly and the composed pipeline would appear to achieve more than it does. The union is therefore the only figure that characterises the pipeline as a whole, and any excess of the summed per-layer counts over the union is redundancy, not additional coverage.

Two structural constraints bound what this attribution can show, and both suppress apparent complementarity rather than inflate it. The repository-edge layer detects hardcoded credentials and comparable textual patterns; it is insensitive to every control expressed as a resource attribute, so on a corpus dominated by attribute-level controls its contribution is necessarily small. The plan-level layer, conversely, can only be credited on cases for which a plan was produced; cases whose planning failed are excluded from its denominator and are not recorded as misses. The native module-testing layer validates the project's own infrastructure and is not exercised per benchmark case, so it is excluded from this table and reported under RQ3 instead. Its omission is a scope boundary, not a null result.

#### E. RQ5: External Generalization

Generalization is assessed only on subsets whose configurations are present in the replication package. The distinction matters here because the external manifests enumerate substantially more entries than there are configurations on disk, and a manifest entry without a

configuration is a citation, not a case. Counting such entries would inflate the external corpus by roughly two orders of magnitude while contributing no measurement. Section V-A reports the declared and present counts side by side for this reason.

The external configurations that are present are drawn from published CIS Amazon Web Services Benchmark examples and carry ground-truth labels derived from the benchmark control each example illustrates; they enter the admissible corpus on the same terms as the internal cases and are validated by the same procedure.

One defect in how they were stored is worth reporting, because its signature is invisible in aggregate. A case is scanned as a directory rather than as a file, and these four configurations initially shared one directory. Every tool therefore received identical input for all four cases and returned a byte-identical finding set for each, while the scoring stage compared that one result against four different controls. Four rows entered each confusion matrix where one observation existed. This does not add noise so much as misstate precision. Interval width is driven by the number of independent observations, so duplicated cases narrow every interval that includes them without contributing evidence. Nothing in the aggregate output distinguishes this from four genuinely independent cases that happen to agree. Each configuration now occupies its own directory, and the corpus loader refuses to load a corpus in which two cases share one, instead of scoring it.

The defect bears on how the external subset should be read, and the same trap applies to any evaluation whose unit of analysis is a directory. Case isolation has to be enforced, never assumed. Their number is small enough that a per-subset recall estimate would carry an interval spanning most of the unit interval, so we report them as part of the combined admissible corpus and treat the external subset as a check that the harness operates on configurations it did not generate, rather than as an independent effectiveness estimate. We make no claim that effectiveness measured here transfers to production infrastructure; establishing that requires a labelled corpus of real-world configurations, which to our knowledge does not exist for this domain and which we identify as the principal obstacle to progress in Section X.

## VII. Discussion

Two findings from building the harness bear on how IaC security tooling should be evaluated.

First, the matching criterion dominates the headline number. Because native rule identifiers carry no shared semantics, any comparison implicitly chooses what counts as a correct detection, and that choice moves recall by more than the differences between tools typically reported in the literature. A comparison that does not state its matching criterion is not reproducible, whatever its corpus size. This restates Habib and Pradel's finding for IaC [31], and parallels the per-category reporting discipline the SAST literature arrived at for the same reason [33].

Second, taxonomy coverage is a bias mechanism. When a canonical taxonomy is authored around one tool's rule set, findings that other tools emit under identifiers the taxonomy lacks are scored as misses. The effect is systematic and favours the authoring tool. Auditing observed rule identifiers against the taxonomy after execution, and reporting the unmapped remainder, is the minimum control; during this work the audit identified rules from comparator tools that the initial taxonomy omitted and that would otherwise have been scored as failures.

A third observation concerns plan-level evaluation. Because the compiled plan is produced after expression resolution, policies observe concrete values and avoid a class of interpolation-related imprecision that source-level analysis must otherwise reason about [26]. The cost is that plan generation dominates the layer's latency and requires a working provider configuration, so the comparison between source-level and plan-level evaluation is a trade-off, not a verdict on which is better.

## VIII. Threats to Validity

### A. Construct Validity

The controlled corpus was authored by the same investigator who implemented the evaluated policy set. Ground-truth labels therefore encode the same reading of each control that the policies encode, and agreement between the two is not independent evidence of detection capability. This is the most serious limitation of the evaluation, and it is only partly mitigated.

Two mitigations apply. Labels derive from the text of the originating control rather than from any policy implementation, and both the specifications and the generator that consumes them are released, so a reader can audit the derivation instead of taking it on trust. And generalization is assessed on configurations the investigator did not author.

A third check bears on it partially. The blind relabelling pass of Section V-C agreed with 47 of 48 recorded labels, and the one disagreement it surfaced was a genuine specification defect of exactly the predicted kind: a control whose stated text was weaker than the requirement its label encoded, where the stricter reading matched the reference policy set. That the pass found such a case is evidence the mechanism is real, not merely conceivable.

It is not, however, a sufficient mitigation, and we do not present it as one. The second rater is automated, so no claim of human inter-rater reliability is made, and its reading of control texts derives from the same public documentation the specifications were written from, so the two raters share provenance and agreement is biased upward. A single investigator still wrote every specification. The external subset is the only part of the corpus outside this failure mode entirely, and it is too small to estimate effectiveness on its own. Independent relabelling by a second human who has not read the policy set remains the correct mitigation and remains outstanding.

Results on the controlled corpus should therefore be read as an upper bound on field performance. A high score on a developer-authored corpus is at least as consistent with insufficient corpus difficulty as with strong detection, and we do not interpret it as the latter.

### B. Internal Validity

Comparator tools ran with default rule sets, whereas the plan-level policies were authored with knowledge of the corpus taxonomy. This asymmetry favours the plan-level layer. It is disclosed rather than adjusted for, because equivalent tuning effort across heterogeneous engines cannot be operationalised objectively; readers should treat the comparison as bounding what default-configuration scanning achieves, not as a ranking of the tools' attainable capability.

One plan-level policy was corrected after its results had been inspected, and the correction is disclosed here because it is exactly the iteration the comparators did not receive. The audit-log-encryption rule tested for the presence of a key-identifier attribute in the plan document. Terraform serialises three configuration states into that document: an attribute set to a literal carries its value; an attribute set from a reference to another resource is omitted and recorded as not-yet-known; and an attribute that is not set at all is present with a null value. A presence test therefore fails on the configuration that omits the attribute and succeeds on the configuration that sets it from a reference, which inverts the control, not merely weakens it. The compliant case was reported as violating and the violating case as compliant, and the two errors concealed each other in the aggregate. The confusion matrix showed one false positive and one false negative, a signature indistinguishable from ordinary imprecision.

The corrected rule distinguishes not-set from not-yet-known and reports only the former. The latter is genuinely indeterminate at plan time, because the attribute is set but the plan does not record the value it is set to, and it is therefore reported as neither compliant nor violating. This is a property of plan-level evaluation and not of this implementation: any policy engine reading plan documents inherits the same three-way distinction, and any that collapses it will invert the same class of control. We report it as a finding about the evaluation layer rather than as a defect that has been repaired, and note that the same defect class does not affect block-typed attributes, whose unconfigured form is absent from the plan and can be tested for presence safely.

Because this correction was informed by inspecting corpus results, the plan-level figures should be read as reflecting one round of iteration that Checkov and tfsec did not receive. The pre-correction and post-correction results are both in the replication package.

Latency was measured on a single host with recorded specifications. Absolute figures do not transfer across environments. Latency additionally measures each tool at its own interface: the plan-level figure covers policy evaluation

over an already-compiled plan document and excludes the terraform init and terraform plan invocations that produce it, which dominate that layer’s true wall-clock cost in continuous integration. The source-level scanners have no comparable preparation step. Plan-level and source-level latencies are therefore not interchangeable, and the plan-level figure is a lower bound on deployed cost.

### C. External Validity

The corpus targets Terraform against a single cloud provider, and the native module-testing layer is exercised against one project’s infrastructure. Neither the reported effectiveness nor the suite-size measurement of Section VI-C generalises to other IaC technologies, other providers, or the multi-module configurations typical of large organisations. No claim is made about organisational factors such as policy ownership, escalation workflow, or developer friction, because this study measures tool output and does not observe practitioners.

The tool set bounds generalisation further. Three third-party scanners are measured but only two independent rule sets are represented (Section V-F), so the comparison speaks to those rule sets, not to source-level IaC scanning in general. Provider coverage bounds it again: security-policy adoption in Terraform differs materially across cloud providers [7], and this corpus targets one.

### D. Scope

The corpus is deterministic, not adversarial. Reported effectiveness characterises performance on a published taxonomy of known misconfiguration classes. No claim is made regarding novel or deliberately obfuscated insecure configurations, or an adversary who has read the policy set.

## IX. Replication Package

The replication package contains the benchmark corpus with ground-truth annotations and the generator specifications from which those annotations derive; the canonical control taxonomy as auditable data; the per-case output of the blind relabelling pass of Section V-C, with its reasons and its stated limitations; the execution harness, which records raw tool output, resolved versions, and repeated latency samples; the normalization engine; the statistical implementation, with a test suite pinning each method to independently derived values; and the analysis stage that generates every table in this paper. It also retains the measured results from before the plan-level policy correction disclosed in Section VIII-B, so that the disclosure can be checked rather than taken on trust.

Two properties are worth stating explicitly. The harness declines to emit results when no case is admissible, instead of producing an empty but plausible table. And the statistical implementation refuses to report a proportion whose denominator is zero, returning a not-estimable marker instead of a value a reader could mistake for a measurement.

|                       |                         |
|-----------------------|-------------------------|
| benchmark/            |                         |
| benchmark.json        | case catalogue + labels |
| generate_corpus.py    | case generator          |
| internal/             | authored cases          |
| external/             | CIS-derived cases       |
| labelling/            | blind second pass       |
| evaluation/           |                         |
| corpus.py             | admissibility gate      |
| normalize.py          | finding normalization   |
| control_map.json      | canonical taxonomy      |
| stats.py              | exact CI + McNemar      |
| analyze.py            | table generation        |
| security_framework/   |                         |
| policies/*.rego       | plan-level policy set   |
| engine/               | layer 1 scanning        |
| infrastructure/       |                         |
| tests/*.tftest.hcl    | native module suite     |
| results/              |                         |
| raw/                  | per-case tool output    |
| run_manifest.json     | resolved versions       |
| tables/               | generated tables        |
| pre_opa_polarity_fix/ | pre-correction run      |
| experiments/          |                         |
| run_baselines.sh      | full re-measurement     |

Fig. 3. Replication package layout. Ground-truth labels, the case corpus, and the policy sources are separate top-level artefacts, so an independently authored label set can be substituted without modifying the harness. The pre\_opa\_polarity\_fix directory retains the measurements taken before the policy correction disclosed in Section VIII-B.

Fig. 3 gives the package layout. Ground-truth labels, the scenario corpus, and the policy sources are separate top-level artefacts, so a reader can substitute an independently authored label set without modifying the harness.

<https://github.com/mchittineni/IaCSecBench>  
<https://doi.org/10.5281/zenodo.21645016>

## X. Conclusion

This paper contributes an evaluation methodology for IaC security validation tools: a finding-normalization scheme that makes heterogeneous scanner output commensurable, an explicit treatment of matching strictness, a mechanically enforced corpus admissibility procedure, and a statistical protocol suited to paired small-sample comparison. The accompanying framework composes repository-edge validation, native module testing, and plan-level policy evaluation, and is released with a harness in which every reported figure is a generated artefact traceable to recorded tool output.

The methodological findings are the transferable result. Matching criteria and taxonomy coverage exert more influence on reported detection rates than the differences between tools that such reports typically claim to establish, which suggests that comparative claims in this area should be accompanied by both, and that corpus size is a weaker indicator of evidential strength than admissibility and interval width [35].

One outcome bears restating because it is the strongest available evidence that the methodology resists the bias it was built to control. Applied to the pipeline this paper contributes, the procedure reports that a third-party scanner attains the highest control-level recall on

the corpus and that the pipeline's scored layers do not lead the comparison. A benchmark whose own reference implementation places behind a comparator, reported without adjustment, is the property we would ask of any evaluation in this area, including one we did not author.

Future work should extend the taxonomy to IaC technologies beyond Terraform, enlarge the compliant-baseline portion of the corpus so that false-positive behaviour becomes estimable, and evaluate whether plan-level policy evaluation retains its advantage on configurations using deeply nested modules.

### Acknowledgment

The author thanks the maintainers of Terraform, Open Policy Agent, Checkov, and tfsec. AI-assisted tooling was used for manuscript copy-editing; all technical content, measurements, and conclusions are the author's.

### References

- [1] K. Morris, *Infrastructure as Code: Dynamic Systems for the Cloud Age*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [2] M. Guerriero, M. Garriga, D. A. Tamburri, and F. Palomba, "Adoption, support, and challenges of infrastructure-as-code: Insights from industry," in *Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME)*, 2019, pp. 580–589.
- [3] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, "A view of cloud computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.
- [4] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proc. 41st Int. Conf. Software Engineering (ICSE)*, Montreal, QC, Canada, 2019, pp. 164–175.
- [5] A. Rahman, M. R. Rahman, C. Parnin, and L. Williams, "Security smells in Ansible and Chef scripts: A replication study," *ACM Trans. Software Engineering and Methodology*, vol. 30, no. 1, pp. 1–31, 2021.
- [6] A. Verdet, M. Hamdaqa, L. Da Silva, and F. Khomh, "Exploring security practices in infrastructure as code: An empirical study," *arXiv preprint arXiv:2308.03952*, 2023.
- [7] —, "Assessing the adoption of security policies by developers in Terraform across different cloud providers," *Empirical Software Engineering*, vol. 30, no. 2, 2025.
- [8] L. Bass, I. Weber, and L. Zhu, *DevOps: A Software Architect's Perspective*. Boston, MA, USA: Addison-Wesley, 2015.
- [9] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA, USA: Addison-Wesley, 2010.
- [10] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.
- [11] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [12] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.
- [13] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [14] A. Rahman, E. Farhana, C. Parnin, and L. Williams, "Gang of eight: A defect taxonomy for infrastructure as code scripts," in *Proc. 42nd Int. Conf. Software Engineering (ICSE)*, 2020, pp. 752–764.
- [15] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source Kubernetes manifests: An empirical study," *ACM Trans. Software Engineering and Methodology*, vol. 32, no. 4, pp. 1–36, 2023.
- [16] M. S. I. Shamim, F. A. Bhuiyan, and A. Rahman, "XI commandments of Kubernetes security: A systematization of knowledge related to Kubernetes security practices," in *Proc. IEEE Secure Development Conf. (SecDev)*, 2020, pp. 58–64.
- [17] S. Dalla Palma, D. Di Nucci, and D. A. Tamburri, "Toward a catalog of software quality metrics for infrastructure code," *Journal of Systems and Software*, vol. 170, p. 110726, 2020.
- [18] S. Dalla Palma, D. Di Nucci, F. Palomba, and D. A. Tamburri, "Within-project defect prediction of infrastructure-as-code using product and process metrics," *IEEE Trans. Software Engineering*, vol. 48, no. 6, pp. 2086–2104, 2022.
- [19] M. Chiari, M. De Pascalis, and M. Pradella, "Static analysis of infrastructure as code: A survey," in *Proc. IEEE 19th Int. Conf. Software Architecture Companion (ICSA-C)*, 2022, pp. 218–225.
- [20] N. Saavedra and J. F. Ferreira, "GLITCH: Automated polyglot security smell detection in infrastructure as code," in *Proc. 37th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2022, pp. 1–12.
- [21] N. Saavedra, J. Gonçalves, M. Henriques, J. F. Ferreira, and A. Mendes, "Polyglot code smell detection for infrastructure as code with GLITCH," in *Proc. 38th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2023, pp. 2042–2045.
- [22] A. War, S. L. Nikiema, J. Samhi, J. Klein, and T. F. Bissyandé, "Security smells in infrastructure as code: A taxonomy update beyond the seven sins," *arXiv preprint arXiv:2509.18761*, 2025.
- [23] H. Roe, M. Gogate, and K. Dashtipour, "Multicloud security assessment: A benchmark study of infrastructure as code scanners," *ICCK Trans. Information Security and Cryptography*, vol. 2, no. 2, pp. 109–118, 2026.
- [24] A. B. Gokhale, V. Jayaprakash, and N. Singaravelu, "Comparative analysis of infrastructure-as-code misconfiguration detection through policy driven evaluation and taint-aware static reasoning," in *Proc. Int. Conf. (Lecture Notes in Computer Science)*, 2026, pp. 400–413.
- [25] I. A. Kapetanidou, A. Nizamis, and K. Votis, "An evaluation of commonly used Kubernetes security scanning tools," in *Proc. ACM Int. Conf. (companion)*, 2025, pp. 20–25.
- [26] R. Opdebeeck, A. Zerouali, and C. De Roover, "Control and data flow in security smell detection for infrastructure as code: Is it worth the effort?" in *Proc. 20th IEEE/ACM Int. Conf. Mining Software Repositories (MSR)*, 2023, pp. 534–545.
- [27] —, "Smelly variables in Ansible infrastructure code: Detection, prevalence, and lifetime," in *Proc. 19th Int. Conf. Mining Software Repositories (MSR)*, 2022, pp. 61–72.
- [28] Y. Qiu, P. T. J. Kon, R. Beckett, and A. Chen, "Unearthing semantic checks for cloud infrastructure-as-code programs," in *Proc. 30th ACM Symp. Operating Systems Principles (SOSP)*, 2024, pp. 574–589.
- [29] F. Minna, A. Blaise, K. Tuma, and F. Massacci, "Automated analysis of security policy violations in Helm charts," *IEEE Trans. Dependable and Secure Computing*, vol. 23, no. 2, pp. 2519–2533, 2026.
- [30] P. L. Foalet, L. D. Silva, F. Khomh, and E. Merlo, "An empirical study of policy-as-code adoption in open-source software projects," *Journal of Systems and Software*, vol. 242, p. 113028, 2026.
- [31] A. Habib and M. Pradel, "How many of all bugs do we find? a study of static bug detectors," in *Proc. 33rd IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2018, pp. 317–328.
- [32] OWASP Foundation, "OWASP benchmark project," <https://owasp.org/www-project-benchmark/>, 2023, accessed Aug. 2, 2026.
- [33] K. Li, S. Chen, L. Fan, R. Feng, H. Liu, C. Liu, Y. Liu, and Y. Chen, "Comparison and evaluation on static application security testing (sast) tools for Java," in *Proc. 31st ACM Joint European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE)*, 2023, pp. 921–933.
- [34] M. Esposito, V. Falaschi, and D. Falessi, "An extensive comparison of static application security testing tools," in *Proc. 28th Int. Conf. Evaluation and Assessment in Software Engineering (EASE)*, 2024, pp. 69–78.
- [35] M. Böhme, L. Szekeres, and J. Metzger, "On the reliability of coverage-based fuzzer benchmarking," in *Proc. 44th Int. Conf. Software Engineering (ICSE)*, 2022, pp. 1621–1633.

- [36] Open Policy Agent, “Open Policy Agent documentation,” <https://www.openpolicyagent.org/docs>, Cloud Native Computing Foundation, 2024, accessed Aug. 2, 2026.
- [37] HashiCorp, “Terraform tests,” <https://developer.hashicorp.com/terraform/language/tests>, 2024, accessed Aug. 2, 2026.
- [38] Center for Internet Security, “CIS Amazon Web Services foundations benchmark, v3.0.0,” [https://www.cisecurity.org/benchmark/amazon\\_web\\_services](https://www.cisecurity.org/benchmark/amazon_web_services), 2024, accessed Aug. 2, 2026.
- [39] National Institute of Standards and Technology, “Security and privacy controls for information systems and organizations,” NIST, Tech. Rep. Special Publication 800-53, Rev. 5, 2020.
- [40] C. Vassallo, S. Panichella, F. Palomba, S. Proksch, H. C. Gall, and A. Zaidman, “How developers engage with static analysis tools in different contexts,” *Empirical Software Engineering*, vol. 25, no. 2, pp. 1419–1457, 2019.
- [41] M. Tahaei, K. Vaniea, K. K. Beznosov, and M. K. Wolters, “Security notifications in static analysis tools: Developers’ attitudes, comprehension, and ability to act on them,” in *Proc. ACM CHI Conf. Human Factors in Computing Systems*, 2021, pp. 1–17.
- [42] X. Ge, C. Fang, X. Li, W. Sun, D. Wu, J. Zhai, S.-W. Lin, Z. Zhao, Y. Liu, and Z. Chen, “Machine learning for actionable warning identification: A comprehensive survey,” *arXiv preprint arXiv:2312.00324*, 2023.
- [43] A. Shostack, *Threat Modeling: Designing for Security*. Indianapolis, IN, USA: John Wiley & Sons, 2014.
- [44] A. Agresti and B. A. Coull, “Approximate is better than “exact” for interval estimation of binomial proportions,” *The American Statistician*, vol. 52, no. 2, pp. 119–126, 1998.
- [45] B. W. Matthews, “Comparison of the predicted and observed secondary structure of T4 phage lysozyme,” *Biochimica et Biophysica Acta (BBA) – Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [46] A. Arcuri and L. Briand, “A hitchhiker’s guide to statistical tests for assessing randomized algorithms in software engineering,” *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219–250, 2014.

Manideep Chittineni is a Senior Platform, Cloud and DevOps Engineer based in the United Kingdom. His work concerns cloud platform automation, infrastructure security validation, and continuous delivery engineering.